Team #88 — M2: Mid-submission

Problem Statement 1: AI Incident Report Assistant (Qualcomm QIDK)

1. Current approach and changes since M1

The core idea is unchanged: an offline Android assistant where **the model proposes and the code verifies**. Speech is transcribed, Qwen3-4B (w4a16, Genie/QAIRT 2.45, Hexagon V79 NPU) extracts the report fields as JSON, and deterministic Kotlin decides everything that affects the final report: grounding, severity, which follow-up to ask and when to stop. Everything runs on the QIDK (Snapdragon 8 Elite, `SM8750P`, Android 15). The app has no `INTERNET` permission.

Current architecture (Figure 1): the LLM only turns language into fields; every decision that shapes the report or the conversation is deterministic Kotlin.

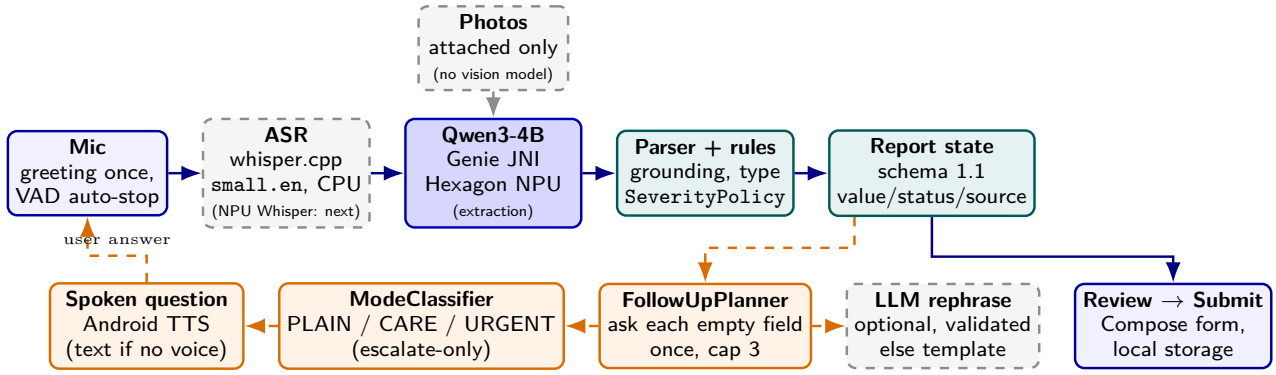


Figure 1: Current architecture. Blue: models on the NPU/CPU; green: deterministic Kotlin rules; orange: follow-up dialogue (dashed arrows: spoken loop); grey: stopgap or partial.

Area	M1 plan	Now (and why)
App shell	Qualcomm QIDK ASR-LLM-TTS sample	**Own native Kotlin / Jetpack Compose (MVVM) app** with our own JNI wrapper around Genie (`genie_jni.cpp`). Qwen3-4B is loaded once at start and the conversation is reset before each call. This gave us full control of the form, dialogue and review UI.
Speech-to-text	Whisper-Small multilingual on the NPU	**CPU stopgap**: whisper.cpp `small.en`, 6 threads, English only. No Whisper NPU binaries were available for our QAIRT version. NPU ASR is still the target (Gemma-4-E2B audio input is being evaluated as an alternative).
Follow-up questions	Template questions; each field asked at most twice	**Follow-up dialogue v2**: one fixed greeting, then questions *only* for empty fields; **each field asked at most once** (no retry or "could you repeat" prompts); silent, unsure or unclear answers become `no_response` / `unknown_reporter_unsure` / `unverified` and are highlighted on the review screen. Tone modes `PLAIN` / `CARE` / `URGENT` (escalate-only, with a negation guard for "no one was hurt"). **Hard cap of 3 questions per report.** Hands-free answers: the mic starts after each question and stops after ∼2 s of silence.
Question wording	Fixed templates	Templates by default; optional LLM rephrase (one per question) that must pass a validator: one, ≤20 words, no "why", no blocked phrases, ≤1.5 s. Otherwise the template is used.
Severity	5-tier rulebook (OSHA / WHO / KABCO)	Deterministic `SeverityPolicy`: final severity = max(model hint, rule floors). `URGENT` ≥ High; `URGENT` + escalation cue ("trapped", "bleeding badly") or injury = Critical; fire, gas, electrical ≥ High. User edits are never overwritten. Designed next: a separate **urgency** axis (U0–U3) with hard safety rules that can only raise severity and urgency.
Emergency	10 s countdown, then alert contacts	One spoken safety check and an emergency banner. After submission, a call-for-help card opens the dialer (`ACTION_DIAL`, no call permission). The read-back never claims "help is on the way" unless an alert was actually sent.
Photo cues	CLIP hazard labels on the NPU	Photos are attached to the report; no vision model runs yet.

Report schema 1.1: each field stores `value / confidence / source / status / raw_answer`, plus `dialogue{mode, asked_fields, emergency}`. The review form has 12 fields in 4 groups, with AI-filled and edited badges.

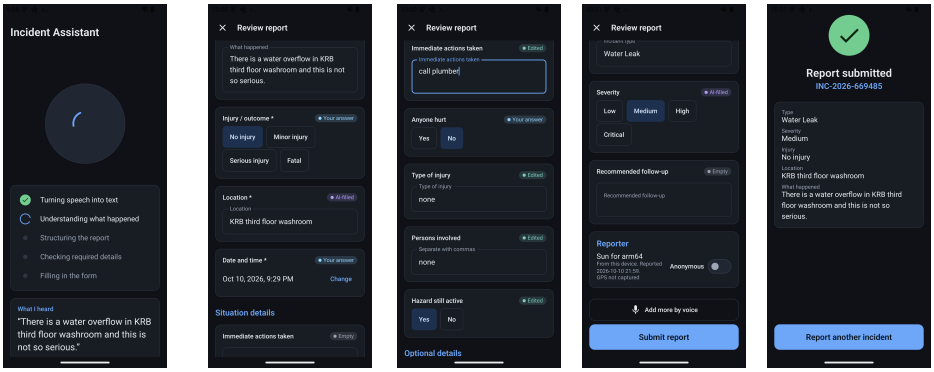
2. Experiments run so far and their results

E1. LLM selection (35-case typed development set, 7 categories, on the QIDK NPU, same harness):

Model	Valid JSON	Type	Severity	Location	Date/time	Overall	Halluc.
Llama 3.2 1B (w4a16)	68.6%	40.0%	48.6%	57.1%	45.7%	47.9%	8
Qwen3-4B-Instruct-2507 (w4a16)	**100%**	**88.6%**	**85.7%**	**100%**	**94.3%**	**92.1%**	**2**
Qwen3-4B + Kotlin rule layer	100%	100%	100%	100%	100%	**100%**	**0**

The rule layer (grounding with snap-back to the user's words, type grounding, injury/symptom rules with negation, hazard floors) raised raw accuracy from 93.6% to 100%. These rules were written after seeing this set, so the figure is optimistic; a held-out set is being written by a team member who has not seen the rules.

E2. Follow-up/correction prompt: 5/6 fully correct (simple answer, fillers, correction of a different field, "I don't know" → empty update, Hinglish). The missed case (symptoms in an answer) is now handled by a code rule. The test also exposed the model repeating the pre-filled `{`; the parser now handles it.



(a) Processing stages (b) Review: AI-filled vs. answered fields (c) User edits marked "Edited" (d) Severity, submit (e) Submitted

Figure 2: The app running on the QIDK (real Qwen3-4B on the NPU, Whisper CPU): one spoken sentence becomes a filled, reviewable report.

E3. Multilingual. Hinglish: 6/6 correct with rules. Romanised Telugu: weak (fire, gas and injury returned as "no incident"), so we added a keyword safety net (e.g. *nippu, debba, aag, chot*). Whisper on a laptop (quality check only): Hindi *transcribe* is accurate, but *translate* loses detail ("the AC in the server room broke" → "the server has been damaged"); Telugu *transcribe* gives unrelated text in all 4 clips, and *translate* keeps the gist in 2 of 4 ("gas smell" → "gas stove").

E4. On-device latency (QIDK, in-app, model loaded once):

Stage	Measured
Qwen3-4B model load	3.1–3.7 s
Extraction (∼1,660-char prompt, 24 runs)	prefill median 758 ms (p90 1,028 ms); decode median 6.9 s (p90 7.7 s)
Update prompt / rephrase prompt	∼0.5 s + 1.6 s / ∼0.5 s + 1.2 s (rephrase is over its 1.5 s budget)
Whisper `small.en`, CPU, 6 threads	∼3.1 s per clip (RTF 0.52), WER 0% on 4 clips; `base.en`: 11.1% WER at ∼1.0 s
End of speech → filled form (Whisper CPU + Qwen NPU)	**12.3 s median, 16.0 s p90**

E5. First end-to-end audio run (real Qwen3-4B + Whisper CPU, 4 human recordings + 1 silent case): ASR WER 0.0%; location 4/4, incident type 2/3, date/time 0/2 (overall field accuracy 66.7%); placement rate 100% (no value in the wrong field); 1.0 question per report, **0 ask-once violations**, 0 invariant violations, flow correct 5/5; tone mode correct 3/4 (one PLAIN report classed `URGENT`); rephraser validator pass rate 40% (3 timeouts), so the templates were used.

3. Progress on testing

- **375 JVM unit tests pass** (no device needed): flow reducer, validation and severity policy, mode classifier and negation guard, follow-up planner (every priority rule, ask-once, caps), phrase validator, 36 model-output mapping tests, 18 parser tests, prompt budget, and 94 tests for the evaluation metrics (WER, field matching, mapping, dialogue invariants, p90).
- **Dialogue scenario tests:** all 14 spec scenarios pass with fake speech input/output (template-only), and again with 6 fake rephrasers (good text, "why", two questions, "let me ask again", null, timeout); cases (b)–(f) correctly fall back to the template. After every scenario we check: no field asked twice, no line spoken twice, mode never decreases.
- **Rule layer self-check:** 24/24 checks pass on real model outputs captured on the device (extraction, Telugu safety net, follow-up loop, parser edge cases).
- **On-device tests:** `GenieLlmBenchmarkTest`, `WhisperCppBenchmarkTest` and `EvalAudioRunTest`. A "Run eval set" item in the developer menu writes `results.json`; a script turns each run into a results table.
- **Test sets:** (a) 35-case typed development set; (b) 30-case English end-to-end eval set (26 scripted + 4 real recordings), only 5 with audio so far; (c) a new **office severity benchmark**: 30 cases (27 incidents, 3 non-incidents) in **6 languages** (English, Hindi, Assamese, Odia, Telugu, Tamil; 180 narrations with Roman transliterations), with gold incident type, severity 0–4, urgency U0–U3, status, emergency flag, expected fields and known traps (e.g. "damage" that is only cosmetic, a rash without airway symptoms). Not yet run against the model.
- **Not yet done:** airplane-mode run; held-out results; audio for 25 eval cases; TTS output (the board has no TTS engine installed, so the app shows the text instead).

4. Future plans

1. **Multilingual:** Hindi, Telugu and code-mixed speech in (Whisper translate mode, native-reviewed question banks), with report fields kept in English.
2. **Image support:** attach scene photos to the report and add an on-device vision model that suggests the incident type and severity.
3. **Emergency calling:** in an emergency, call the listed contact numbers immediately (cancellable countdown first, never silent).
4. **Incident map:** a maps SDK view showing all current incidents with their locations and severity to the user.
5. **Voice activation** from the home screen, so a report can start hands-free.
6. **Default vs. customisable form fields:** a standard field set, plus organisation-defined fields.
7. **Dynamic follow-up questions:** generated from the report context, once the basic design is stable.
8. **Summary and verification:** read a summary of the incident back to the user at the end (on screen and spoken by TTS) so they can verify it.